



CEBU INSTITUTE OF TECHNOLOGY
U N I V E R S I T Y

IT342-G3

System Integration and Architecture

System Design Document (SDD)

Project Title: Grossery

Prepared By: Lugatiman, Simon Jay D.

Version: 1

Date: 4/4/26

Status: Approved

REVISION HISTORY TABLE

Version	Date	Author	Changes Made	Status
0.1	2/21/26	Lugatiman, Simon Jay D.	Initial draft	Draft
0.2	3/02/26	Lugatiman, Simon Jay D.	Added API specifications	Review
0.3	3/17/26	Lugatiman, Simon Jay D.	Updated database design	Review
0.4	3/31/26	Lugatiman, Simon Jay D.	Added UI/UX designs	Review
0.5	3/31/26	Lugatiman, Simon Jay D.	Incorporated feedback	Revised
0.6	4/2/26	Lugatiman, Simon Jay D.	Final review and corrections	Final
1	4/4/26	Lugatiman, Simon Jay D.	Baseline version for development	Approved

TABLE OF CONTENTS

Contents

EXECUTIVE SUMMARY	4
1.0 INTRODUCTION	5
2.0 FUNCTIONAL REQUIREMENTS SPECIFICATION	5
3.0 NON-FUNCTIONAL REQUIREMENTS	8
4.0 SYSTEM ARCHITECTURE	9
5.0 API CONTRACT & COMMUNICATION	10
Authentication Endpoints	11
User Registration	11
User Login	11
6.0 DATABASE DESIGN	13
7.0 UI/UX DESIGN	14
8.0 PLAN	17

EXECUTIVE SUMMARY

1.1 Project Overview

Grossery is a grocery consumption planning and tracking application designed to help users manage household grocery usage by comparing expected monthly consumption with actual consumption. The system allows users to define grocery items, set expected usage values, log real monthly consumption, and view analytics that highlight overconsumption or underconsumption trends. The system includes a Spring Boot backend API, React web application, and Android mobile app, all integrated to provide a seamless tracking experience across platforms.

1.2 Objectives

1. Develop a functional grocery consumption tracking system with secure user authentication
2. Implement a three-tier architecture using Spring Boot (backend), React (web), and Android (mobile)
3. Create RESTful APIs for communication between all system components
4. Design a responsive user interface that works consistently across web and mobile platforms
5. Deploy all system components to production-ready environments
6. Allow users to set expected monthly consumption per grocery item
7. Allow users to log actual consumption monthly for comparison
8. Generate dashboards and charts showing expected vs actual consumption results

1.3 Scope

Included Features:

- User registration and authentication (email/password, BCrypt, and JWT)
- Google OAuth 2.0 social login integration with custom JWT generation
- User profile viewing with Role-Based Access Control (Admin/User)
- Create, edit, delete, and view grocery items (Full CRUD)
- Set expected monthly consumption for grocery items

- Log actual monthly consumption per item with automated variance calculation
- Expiry tracking through manual expiration date entry and automated status indicators
- Receipt file upload and server-side storage linked to grocery records (viewing of receipt images only; it does not perform OCR or automated text parsing)
- External API integration to fetch and display general market price trends
- SMTP email notifications for welcome messages and overconsumption/expiry alerts
- Monthly comparison dashboard showing expected vs actual differences
- Basic chart visualization (horizontal bar chart for expected vs actual)
- Filtering and sorting grocery items by name and consumption status
- MySQL relational database schema with minimum 5 normalized tables

Excluded Features:

- Barcode scanning and receipt scanning
- Online grocery store integration for inventory checking, online ordering, or direct store-to-app price matching
- Automatic grocery recommendations using AI
- Push notifications and SMS reminders
- Shared household accounts (multi-user pantry system)
- Advanced analytics (forecasting algorithms, machine learning)
- Meal planning and recipe suggestions

1.0 INTRODUCTION

1.1 Purpose

This document serves as the comprehensive design specification for the Grossery system. It provides detailed requirements, architectural decisions, API contracts,

database design, and implementation roadmap to guide development and ensure all components integrate seamlessly.

2.0 FUNCTIONAL REQUIREMENTS SPECIFICATION

2.1 Project Overview

Project Name: Grossery

Domain: Grocery Management / Personal Consumption Tracking

Primary Users: Individuals and households

Problem Statement: Many people struggle with grocery budgeting and consumption planning because they do not have clear visibility into whether their grocery usage matches their expectations. This often leads to overspending, shortages, and food waste.

Solution: A grocery consumption planning tool that enables users to set expected monthly usage and compare it against actual monthly consumption through reports and visual analytics.

2.2 Core User Journeys

Journey 1: First-time User Setup and Grocery Planning

1. User visits Grossery web application
2. User clicks "Register" and creates an account
3. User logs in using email and password
4. User navigates to the dashboard
5. User adds grocery items (e.g., rice, eggs, milk)
6. User sets expected monthly consumption per item
7. Dashboard updates expected monthly totals

Journey 2: Monthly Consumption Logging

1. User logs in
2. User selects the current month
3. User enters actual consumption for each grocery item
4. System saves consumption records in the database
5. Dashboard updates monthly reports and graphs

Journey 3: Monthly Review and Adjustment

1. User logs in at the end of the month
2. User views monthly comparison report
3. User identifies items that were overconsumed or underconsumed
4. User updates expected monthly consumption values to improve planning accuracy
5. System stores updated expectations for future months

2.3 Feature List (MoSCoW)

MUST HAVE

1. User authentication (register, login, Google OAuth, logout)
2. User profile page with ROLE_USER and ROLE_ADMIN awareness
3. Grocery item management (Full CRUD: add, update, delete, view)
4. Expected monthly consumption input per item
5. Monthly actual consumption logging
6. Receipt File Upload (linked to grocery record)
7. External API Integration (Real-time market price display)
8. Email Notifications (Welcome email & overconsumption alerts via SMTP)
9. Dashboard summary (expected vs actual comparison)
10. Graph visualization (expected vs actual bar chart)

SHOULD HAVE

1. Filtering grocery items by name or category
2. Monthly consumption history viewing
3. Input validation with clear error messages
4. Responsive design for all screen sizes

COULD HAVE

1. Monthly performance score (accuracy percentage)
2. Export monthly report (CSV)

WON'T HAVE

1. Barcode/receipt scanning
2. Store integration
3. Push notifications
4. Shared household accounts
5. Advanced forecasting and AI recommendations

2.4 Detailed Feature Specifications

Feature: User Authentication & Social Login

- **Screens:** Registration Page, Login Page, User Profile
- **Fields:** First Name, Last Name, Email, Password, Confirm Password
- **Validation:**
 - Email must be a valid format and unique
 - Password must meet minimum security length and is hashed via BCrypt
- **API Endpoints:**
 - POST /api/auth/register
 - POST /api/auth/login
 - GET /api/auth/oauth2/google: Initiates Google OAuth2; the system generates a custom JWT upon successful login
 - GET /api/users/me: Returns current authenticated user data (the required /me endpoint)
- **Security:**
 - JWT Token generation, validation, and secure storage
 - Role-Based Access Control (RBAC): Distinct API and UI-level permissions for ROLE_USER and ROLE_ADMIN

Feature: Grocery Item & Receipt Management (Core Module)

- **Screens:** Grocery List, Add/Edit Item, Receipt Viewer
- **Fields:** Item Name, Category, Unit, Expected Monthly Consumption, Receipt File (Upload)
- **Functions:**
 - Full CRUD: Create, Read, Update, and Delete for the Grocery Item entity
 - File Upload: Upload images (JPG/PNG) of receipts; files are stored on the server and linked to the database record
 - Admin Actions: ROLE_ADMIN can manage global categories and view system-wide data; ROLE_USER is restricted to personal data

- Logic: Variance Calculation * Formula: $\$Variance = Actual - Expected\$$
 - Status Mapping:
 - * Negative Variance: Indicates "Underconsumed" (Savings).
 - * Positive Variance: Indicates "Overconsumed" (Overage).
 - Zero Variance: Indicates "Balanced" consumption.
- **API Endpoints:**
 - GET /api/groceries, POST /api/groceries
 - PUT /api/groceries/{id}, DELETE /api/groceries/{id}
 - POST /api/groceries/{id}/receipts: Multipart file upload for server-side storage

Feature: Expiry Tracking & Market Data

- **Screens:** Grocery List, Item Detail, Dashboard
- **Fields:** expiration_date (Date Picker)
- **Functions:**
 - Status Badging: UI displays color-coded badges based on the expiration_date (e.g., Red for Expired, Yellow for Soon).
 - Automated Alerts: System triggers an SMTP email when an item is within 3 days of expiring.
- **API Endpoints:**
 - GET /api/groceries/expiring-soon: Retrieves a filtered list of items nearing expiration.
 - GET /api/market-prices: Proxy endpoint to consume real-time external market data.

Feature: Notifications & External Integration

- **External API Integration:** System consumes a real public API (e.g., market price or nutrition data) to display meaningful information in the system
- **Email Sending (SMTP):** Console prints are not accepted
- **Account Email:** Automated "Welcome" or verification email sent upon registration
- **API Endpoints:**
 - POST /api/groceries/{id}/consumption
 - GET /api/market-prices: Proxy endpoint to consume external API data

Feature: Dashboard & Analytics

- **Screens:** Dashboard
- **Display:**
 - Horizontal bar chart comparing Expected vs. Actual consumption

- Variance status and progress indicators
- **Role Awareness:** The Mobile UI adjusts to show or hide management tools based on user roles
- **API Endpoints:**
 - GET /api/dashboard/summary
 - GET /api/dashboard/comparison

2.5 Acceptance Criteria

AC-1: Successful User Registration

- Given I am a new user
- When I enter a valid email and strong password
- And confirm password matches
- And click "Create Account"
- Then my account should be created
- And I should be automatically logged in
- And redirected to the dashboard

AC-2: Add Grocery Item

- Given I am logged in
- When I enter a grocery item name and expected monthly consumption
- And click "Save"
- Then the grocery item should appear in my grocery list
- And be visible in the dashboard

AC-3: Log Monthly Consumption

- Given I am logged in
- When I select a grocery item
- And enter actual consumption for the current month
- And save the entry
- Then the system should calculate the variance
- And update the dashboard comparison chart

AC-4: View Dashboard Analytics

- Given I have entered expected and actual consumption
- When I view the dashboard
- Then I should see a horizontal bar chart

- And the difference should correctly reflect overconsumption or underconsumption
- And the status indicator should match the calculated variance

AC-5: Google OAuth Login

- Given I have a valid Google account.
- When I click the "Login with Google" button on the login page.
- Then I should be redirected to the Google authorization screen.
- And Upon successful authorization, a custom JWT should be generated by the backend.
- And I should be redirected to the Dashboard as an authenticated user.

AC-6: Receipt File Upload

- Given I am on the "Add/Edit Grocery Item" or "Item Detail" screen.
- When I select a receipt image (JPG/PNG) and click "Upload" .
- Then the file should be uploaded and stored on the server.
- And the file path should be linked to the specific grocery database record.
- And I should be able to view the uploaded receipt image in the UI gallery.

AC-7: External API Market Data

- Given I am logged into the system and view the Dashboard or Market Price screen.
- When The page loads or I trigger a "Refresh Prices" action.
- Then the system should consume a real public API to fetch current grocery market prices.
- And the fetched data must be displayed meaningfully in the system UI.

AC-8: Automated Email Notifications (SMTP)

- Given I am a new user completing the registration process.
- When my account is successfully created in the system.
- Then I should receive an automated "Welcome" email in my registered inbox.
- And if my actual consumption exceeds my expected monthly limit, a notification email should be triggered.

3.0 NON-FUNCTIONAL REQUIREMENTS

3.1 Performance & Scalability

- **API Latency:** Response time must be ≤ 2 seconds for 95% of requests.
- **Database Efficiency:** MySQL queries executed via Spring Data JPA must complete within 500ms .

- **Concurrency:** Support at least 100 concurrent users without service degradation.
- **Load Times:** Web pages must load within 3 seconds, and mobile cold starts must be ≤ 3 seconds.

3.2 Security & Authentication

- **Identity Management:** Utilize a custom-built Spring Security configuration for user registration and JWT-based session handling .
- **Encryption:** All communications must use HTTPS. Passwords must be hashed using BCrypt (salt rounds = 12) before storage.
- **Protection:** Implement API-level and UI-level role restrictions (Admin vs. Regular User) and ensure users can only access their own grocery data through secure JPA queries .
- **Rate Limiting:** Restrict API calls to 100 requests/minute per IP to prevent abuse.

3.3 Compatibility Requirements

- **Web Browsers:** Latest two versions of Chrome, Firefox, Safari, and Edge.
- **Android:** Must use API Level 34 (Android 14).
- **UI Framework:** Strictly uses XML-Based Layouts
- **Screen Sizes:** Responsive support for Mobile (360px+), Tablet (768px+), and Desktop (1024px+).
- **Operating Systems:** Windows 10+, macOS 10.15+, and Linux Ubuntu 20.04+ for web access.

3.4 Usability Requirements

- **Efficiency:** New users should be able to add their first grocery item and set an expected consumption value within 2 minutes of registration.
- **Accessibility:** WCAG 2.1 Level AA compliance for web and keyboard navigation support.
- **Navigation:** Consistent sidebar/bottom navigation across all platforms for seamless tracking.
- **Mobile Optimization:** Touch targets must be a minimum of 44x44px to ensure ease of use during manual data entry.
- **Error Recovery:** Provide clear error messages for invalid consumption inputs (e.g., negative numbers) with options to correct the entry.

4.0 SYSTEM ARCHITECTURE

4.1 Component Diagram

Technology Stack:

- **Backend:** Java 17, Spring Boot 3.5.x, Spring Security + JWT, Spring Data JPA
- **Database:** MySQL 8.x (Custom-built instance hosted on Railway, strict No-BaaS policy)
- **Web Frontend:** React 18 (Vite), JavaScript, Vanilla CSS (Custom Design System), Axios
- **Mobile:** Android Kotlin, XML-Based Layouts (Strictly No Compose), Retrofit, Room
- **Build Tools:** Maven (Backend), npm (Web), Gradle (Android)
- **Deployment:** Railway (Backend & MySQL), Vercel/Netlify (Web Frontend), APK (Mobile)

5.0 API CONTRACT & COMMUNICATION

5.1 API Standards

Base URL	https://[server_hostname]:[port]/api
Format	JSON for all requests/responses
Authentication	Bearer token (JWT) in Authorization header
Response Structure	{ "success": boolean, "message": string, "data": object null }

5.2 Endpoint Specifications

Authentication Endpoints

User Registration

Description	User Registration
API URL	/api/auth/register
HTTP Request Method	POST
Format	JSON for all requests/responses
Authentication	None
Request Payload	{

	<pre>"email": "user@example.com", "password": "securePassword123", "firstName": "John", "lastName": "Doe" }</pre>
Response Structure	<pre>{ "success": true, "message": "User registered successfully", "data": { "id": 101, "firstName": "John", "lastName": "Doe", "email": "user@example.com", "role": "ROLE_USER" } }</pre>

User Login

Description	User Login
API URL	/api/auth/login
HTTP Method	POST
Format	JSON for all requests/responses
Authentication	None
Request Payload	<pre>{ "email": "user@example.com", "password": "securePassword123" }</pre>
Response Structure	<pre>{ "success": true, "message": "Login successful", "data": { "accessToken": "eyJhbG...", "tokenType": "Bearer", "email": "user@example.com", "firstName": "John", "lastName": "Doe", "role": "ROLE_USER" } }</pre>

Google OAuth2 Login

Description	Authenticates user via Google and generates a custom JWT
--------------------	--

API URL	/api/auth/oauth2/google
HTTP Method	GET
Format	JSON for all requests/responses
Authentication	None
Request Payload	None
Response Structure	<pre>{ "accessToken": "eyJhbG...", "tokenType": "Bearer", "email": "user@example.com", "firstName": "John", "lastName": "Doe", "role": "ROLE_USER" }</pre>

Receipt File Upload

Description	Uploads a receipt image and links it to a grocery record
API URL	/api/groceries/{id}/receipts
HTTP Method	POST
Format	Multipart/form-data (Request), JSON (Response)
Authentication	Bearer Token (JWT)
Request Payload	File (Image/PDF)
Response Structure	<pre>{ "success": true, "message": "Receipt uploaded successfully", "data": { "receiptId": 501, "fileName": "receipt_01.jpg", "fileUrl": "/uploads/receipts/receipt_01.jpg" } }</pre>

External Market Price Integration

Description	Fetches real-time grocery prices from a public external API
API URL	/api/market-prices
HTTP Method	GET
Format	JSON for all requests/responses
Authentication	Bearer Token (JWT)
Request Payload	None

Response Structure	<pre>{ "success": true, "message": "Market prices retrieved", "data": [{ "itemName": "Rice", "currentPrice": 50.00, "unit": "kg", "lastUpdated": "2026-03-30T10:00:00Z" }] }</pre>
---------------------------	--

Analytics Comparison Data

Description	Retrieves expected vs. actual data for dashboard visualization
API URL	/api/dashboard/comparison
HTTP Method	GET
Format	JSON for all requests/responses
Authentication	Bearer Token (JWT)
Request Payload	None
Response Structure	<pre>{ "success": true, "data": { "month": "March", "year": 2026, "items": [{ "name": "Milk", "expected": 4.0, "actual": 3.5, "variance": -0.5 }] } }</pre>

Log Monthly Consumption

Description	Records actual monthly usage and calculates variance against expected values
API URL	/api/groceries/{id}/consumption

HTTP Method	POST
Format	JSON for all requests/responses
Authentication	Bearer Token (JWT)
Request Payload	<pre>{ "month": "March", "year": 2026, "actualConsumption": 5.5 }</pre>
Response Structure	<pre>{ "success": true, "message": "Consumption logged successfully", "data": { "variance": -0.5, "status": "Underconsumed" } }</pre>

Create/Update Category (Admin Only)

Description	Allows administrators to manage global grocery categories
API URL	/api/categories (or /api/categories/{id} for updates)
HTTP Method	POST/PUT
Format	JSON for all requests/responses
Authentication	Bearer Token (JWT) - ROLE_ADMIN Required
Request Payload	<pre>{ "categoryName": "Dairy" }</pre>
Response Structure	<pre>{ "success": true, "message": "Category saved", "data": { "id": 1, "categoryName": "Dairy" } }</pre>

5.3 Error Handling

HTTP Status Codes

- 200 OK - Successful request

- 201 Created - Resource created
- 400 Bad Request - Invalid input
- 401 Unauthorized - Authentication required/failed
- 403 Forbidden - Insufficient permissions
- 404 Not Found - Resource doesn't exist
- 409 Conflict - Duplicate resource
- 500 Internal Server Error - Server error

Error Code Examples

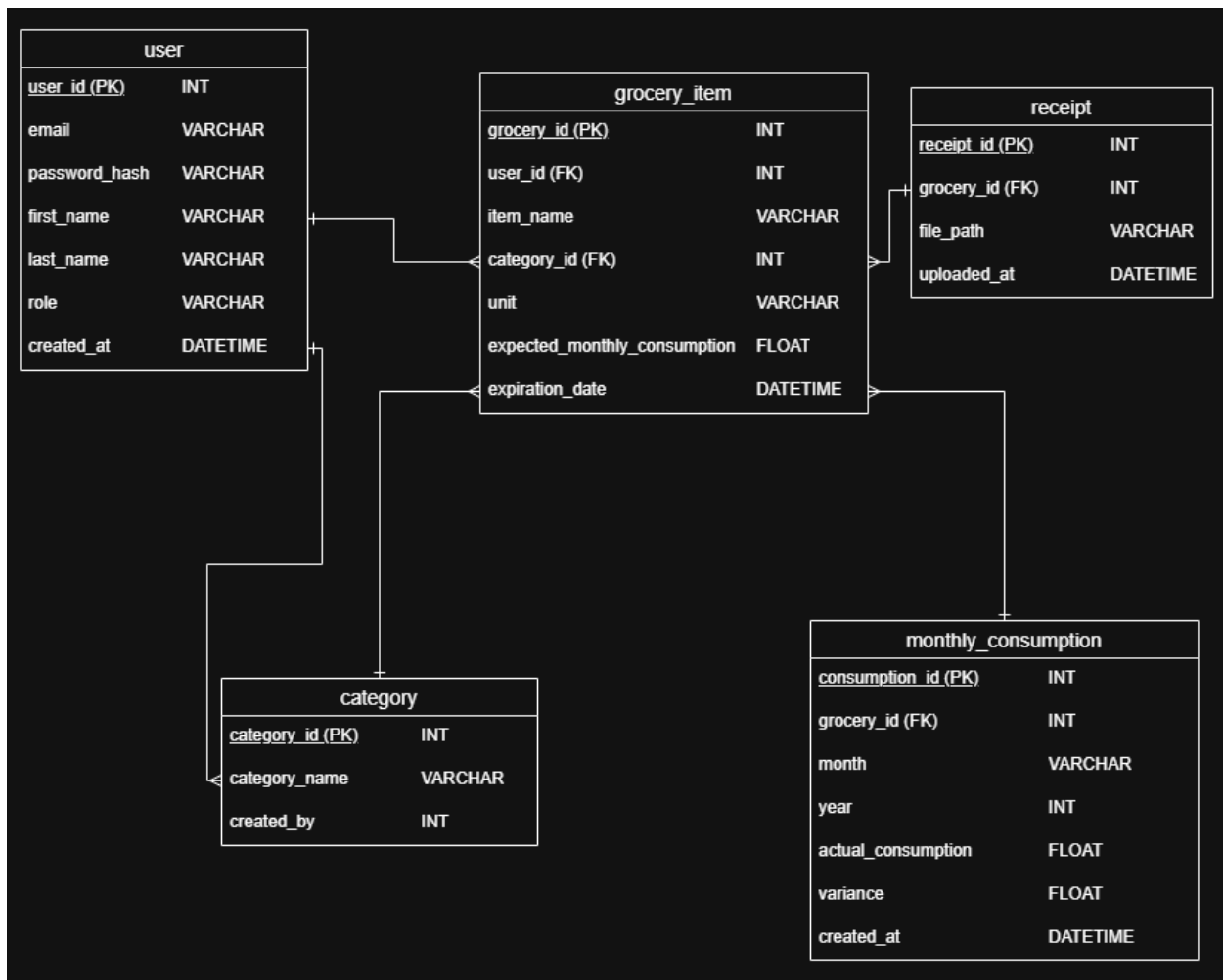
Example 1	<pre>{ "success": false, "message": "Validation failed: First name is required; Email should be valid; ", "data": null }</pre>
Example 2	<pre>{ "success": false, "message": "Authentication failed: Invalid email or password", "data": null }</pre>

Common Error Codes

- Authentication failed: Invalid email or password
- Validation failed: Request payload failed format validation
- Internal error: Unhandled system exception or database failure
- could not execute statement: Database constraint violation (e.g., duplicate email entry)
- User not found after authentication: Account removed during token generation

6.0 DATABASE DESIGN

6.1 Entity Relationship Diagram



7.0 UI/UX DESIGN

7.1 Web Application Wireframes

Grossery

Plan smarter. Waste less. Live fresher

Welcome back

Please enter your details to sign in

EMAIL ADDRESS

Enter your email

PASSWORD

☐ Remember me

[Forgot password?](#)

Sign in

[Don't have an account? Sign up for free](#)

Grossery

Plan smarter. Waste less. Live fresher

Create Account

Start your free Grossery account today

FIRST NAME

Juan

LAST NAME

Dela Cruz

EMAIL ADDRESS

you@example.com

PASSWORD

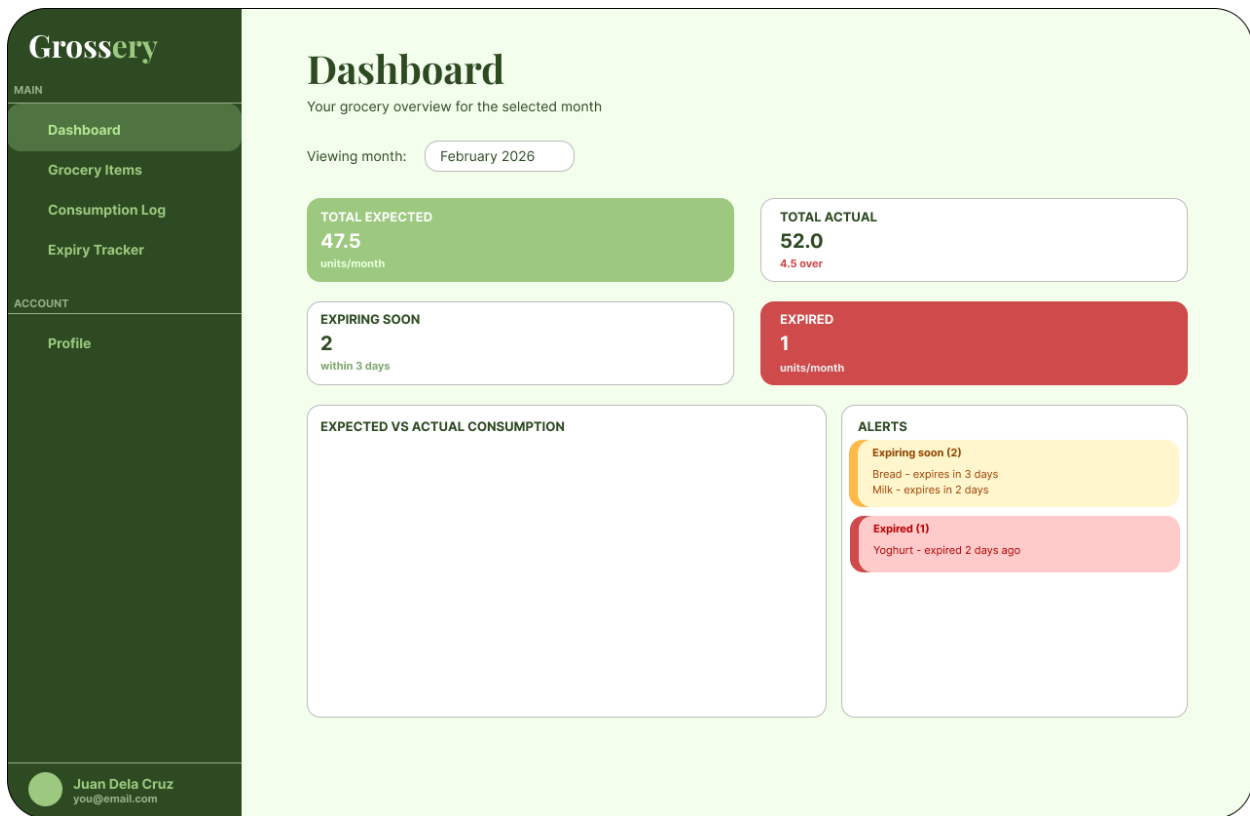
At least 8 characters

CONFIRM PASSWORD

Repeat password

Create account

[Already have an account? Sign in](#)



Main Dashboard (Inventory & Overview)

Header: [Logo] [Search Bar] [Notifications Bell with Expiry Alerts] [User Profile]

Sidebar: [Dashboard] [Monthly Logs] [Analytics] [Settings]

Content: Grocery Inventory Grid (3 columns desktop)

Each Item Card: Name, Category, Unit, Expected Consumption, Days Until Expiration Indicator, "Log Usage" button

Footer: System Status, Copyright, Privacy Policy

Item Detail & History Page

Back Button: Navigation to Main Dashboard

Header Section: Item Name and Unit

Tracking Details: Display of "Expected Monthly Consumption" and "Current Batch Expiration Date"

Consumption History: Table showing Month, Year, Actual Amount, and Variance

Action Buttons: "Edit Item Details" and "Delete Item"

Monthly Consumption Entry (Form/Modal)

Form Title: "Log Monthly Consumption"

Fields: Month Dropdown, Year Input, Actual Consumption Value (numeric), Batch Selection (Closest to Expiry)

Validation: Real-time error messages for negative values or empty fields

Buttons: "Cancel" and "Save Entry"

Analytics & Comparison Dashboard

Chart Section: Horizontal Bar Chart (Expected vs. Actual) for all items

Variance Summary: List of items flagged as "Overconsumed" or "Underconsumed"

Waste Prevention: List of items nearing expiration to prioritize consumption

Filters: View by Category, Variance Status, or Expiration Urgency

User Profile & Settings

Profile Header: User Full Name and Email Address

Account Management: Update Password and Change Unit Preferences

Notification Settings: Toggle alerts for "Low Stock" or "Approaching Expiry" (triggered when an item is ≤ 3 days from its expiration date)

Logout Button: End session and redirect to Login Page

7.2 Mobile Application Wireframes

Bottom Navigation

[Home] [Search] [Add Item] [Profile]

Home Screen

Search Bar: Filter by item name or category

Inventory Feed: 2-column grid layout

Status Badges: Color-coded expiration alerts (Red: Expired, Yellow: Soon)

Interaction: Pull to refresh for real-time backend data sync

Item Detail Screen

Header: Back arrow and uploaded receipt/item image gallery

Tracking Info: Progress bar showing "Actual vs. Expected" consumption

Expiry Info: Countdown timer showing days until next batch expires (if expiration date is ≤ 3 days)

Logging: Fixed bottom button for "Log Current Consumption"

Consumption Logging Flow

Step Indicator: Select Item → Input Amount → Select Expiration Date/Batch → Success

Input: Numeric keypad optimized for quick entry

Variance Preview: Real-time display of how the entry affects monthly status

Submission: Large "Save Entry" button with haptic feedback

Mobile-Specific Features:

- **Touch-optimized:** All buttons and interactive elements maintain a minimum size of 44x44px
- **Offline Caching:** Local Room database stores grocery metadata for offline viewing
- **Bottom Navigation:** Primary actions (Home, Search, Add) positioned for thumb-reach accessibility
- **Simplified Input:** Use of dropdowns and steppers to minimize manual typing on mobile devices

Design System:

- **Colors:** Primary: #9cc97f (Light Green), Background: #f5faf2 (Light Mint), Dark Accents: #2d4a22 (Forest Green), Warning/Expiring: #e8a838 (Yellow/Orange), Error/Expired: #d95f4b (Red)
- **Typography:** Standard Body: DM Sans font family, Display Headers: Playfair Display (Serif)
- **Spacing:** Strict 8px grid system for all padding, margins, and layout alignment
- **Components:** Consistent buttons and cards featuring a 14dp corner radius implemented via XML Shape Drawables (<corners android:radius="14dp" />)
- **Responsive:** Mobile-first approach, breakpoints at 640px, 768px, 1024px

8.0 PLAN

8.1 Project Timeline

Phase 1: Planning & Design (Week 1-2)

Week 1: Requirements & Architecture

Day 1-2: Project setup and documentation

Day 3-4: Complete FRS and NFR

Day 5-7: System architecture design

Week 2: Detailed Design

Day 1-2: API specification

Day 3-4: MySQL database schema design

Day 5-6: UI/UX wireframes for Grossery dashboard

Day 7: Implementation plan finalization

Phase 2: Backend Development (Week 3-4)

Week 3: Foundation

Day 1: Spring Boot setup and MySQL schema design .

Day 2: JPA Entity mapping and Repository layer setup.

Day 3: Custom Spring Security and JWT implementation.

Day 4: Google OAuth2 and SMTP Email service.

Day 5: User and Profile endpoints (/me).

Week 4: Core Features

Day 1: Monthly Consumption Logging functionality

Day 2: Variance Calculation Logic (\$actual - expected\$)

Day 3: Search, filtering, and dashboard data aggregation

Day 4: Error handling and input validation

Day 5: API documentation and integration testing

Phase 3: Web Application (Week 5-6)

Week 5: Frontend Foundation

Day 1: React setup with JavaScript

Day 2: Custom JWT Auth pages (login, register)

Day 3: Grocery List and Inventory dashboard

Day 4: Item Usage Details and history view

Day 5: Consumption Entry Form implementation

Week 6: Complete Web Features

Day 1: Monthly Comparison Analytics (Charts)

Day 2: Consumption Variance Reports and history

Day 3: Admin/User Profile management dashboard

Day 4: Responsive design polish for all screen sizes

Day 5: Full API integration and end-to-end testing

Phase 4: Mobile Application (Week 7-8)

Week 7: Android Foundation

Day 1: Android Studio setup and project structure

Day 2: Retrofit Auth integration

Day 3: Mobile Grocery Browsing and list

Day 4: Quick-Log Consumption interface

Day 5: Retrofit API service layer implementation

Week 8: Complete Mobile App

Day 1: Variance Display and status indicators

Day 2: Monthly Trend Visualization (Mobile Charts)

Day 3: UI polish, touch optimizations, and animations

Day 4: Testing on physical Android devices/emulators

Day 5: APK generation and technical documentation

Phase 5: Integration & Deployment (Week 9-10)

Week 9: Integration Testing

Day 1: End-to-end testing across Web, Mobile, and API

Day 2: Bug fixes and logic optimization

Day 3: Spring Security and RBAC review

Day 4: Performance and concurrency testing

Day 5: Final documentation updates

Week 10: Deployment

Day 1: Backend deployment (Railway)

Day 2: Web app deployment (Vercel/Netlify)

Day 3: Mobile APK distribution

Day 4: Final production testing

Day 5: Final project submission

Milestones:

- **M1 (End Week 2):** All design documents complete
- **M2 (End Week 4):** Backend API fully functional
- **M3 (End Week 6):** Web application complete
- **M4 (End Week 8):** Mobile application complete
- **M5 (End Week 10):** Full system deployed and integrated

Critical Path:

1. Custom JWT & Spring Security (Week 3)
2. Grocery Management API (Week 3-4)
3. Consumption & Variance Logic (Week 4)
4. Analytics & Chart Integration (Week 6)
5. Cross-platform synchronization testing (Week 9)

Risk Mitigation:

- Start with simplest working version of tracking features
- Test integration points early and often
- Keep backup of working versions/database snapshots
- Focus on core consumption logging before UI enhancements